ğrencibilgiLab 06 – Hafta 6Qt Model/View Mimarisi

Lab Hakkında

Bu laboratuvar çalışmasında Qt6'nın Model/View mimarisini uygulamalı olarak öğreneceksiniz. `QStringListModel` ile basit listeden başlayıp, `QAbstractTableModel` ile özel model oluşturmaya, delegeler ile görsel düzenlemeye ve `QSortFilterProxyModel` ile filtrelemeye kadar ilerleyeceksiniz. Tüm örnekler OBYS projesi bağlamındadır.

Süre: 2 saat **Gereksinimler:** Qt6, CMake 3.16+, C++20 destekli derleyici

Qt Projesi Derleme

Tüm çalışmalarda aşağıdaki CMake yapısını kullanın:

```
cmake_minimum_required(VERSION 3.16)
project(Lab06MV LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_AUTOMOC ON)
find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()
qt_add_executable(app main.cpp)
target_link_libraries(app PRIVATE Qt6::Widgets)
```

Derleme komutları:

```
mkdir build && cd build
cmake ..
cmake --build .
```

1 Çalışma 1: QStringListModel ve QListView (30 dk)

Ön Bilgi

Model/View mimarisi, veriyi (model) görünümden (view) ayırarak aynı verinin birden fazla şekilde gösterilmesini sağlar. Qt'da üç temel bileşen vardır:

- **Model** – Veriyi tutar ve erişim sağlar (`QAbstractItemModel` alt sınıfları)
- **View** – Modeldeki veriyi görsel olarak sunar (`QListView`, `QTableView`, `QTreeView`)
- **Delegate** – Veri düzenleme ve özel çizim sağlar (`QStyledItemDelegate`)

`QStringListModel` en basit hazır modeldir: bir `QStringList` üzerinde çalışır ve `QListView` ile birlikte kullanılır.

Aşağıdaki başlangıç kodunu `main.cpp` dosyasına yazın. Önce model ve görünümü oluşturuyoruz:

```
#include <QApplication>
#include <QWidget>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QListView>
#include <QStringListModel>
#include <QPushButton>
#include <QLineEdit>
#include <QLabel>
#include <QMessageBox>
```

```

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QWidget pencere;
    pencere.setWindowTitle("OBYS - Bölüm Listesi");
    pencere.resize(400, 350);

    auto* anaDuzen = new QVBoxLayout(&pencere);

    // Baslik
    auto* baslik = new QLabel("Bölüm Yonetimi");
    baslik->setStyleSheet(
        "font-size: 16px; font-weight: bold;"
        " color: #008080;");
    baslik->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(baslik);
```

Şimdi `QStringListModel` oluşturup `QListView`'a bağlıyoruz:

```
    // Model oluştur
    auto* model = new QStringListModel;
    model->setStringList({
        "Bilgisayar Bilimleri",
        "Yazilim Muhendisligi",
        "Elektrik-Elektronik",
        "Makine Muhendisligi"
    });

    // Görünüm oluştur ve modeli bağla
    auto* liste = new QListView;
    liste->setModel(model);
    liste->setEditTriggers(
        QAbstractItemView::NoEditTriggers);
    anaDuzen->addWidget(liste);
```

Ekleme formu ve butonları oluşturuyoruz:

```
    // Ekleme formu
    auto* formDuzen = new QHBoxLayout;
    auto* yeniAlani = new QLineEdit;
    yeniAlani->setPlaceholderText("Yeni bölüm adi...");
    auto* ekleButon = new QPushButton("Ekle");
    auto* silButon = new QPushButton("Sil");
    formDuzen->addWidget(yeniAlani);
    formDuzen->addWidget(ekleButon);
    formDuzen->addWidget(silButon);
    anaDuzen->addLayout(formDuzen);

    // Durum etiketi
    auto* durumLabel = new QLabel("4 bölüm kayitli");
    durumLabel->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(durumLabel);
```

Son olarak ekleme ve silme bağlantılarını kuruyoruz:

```
    // Ekle butonu
    QObject::connect(ekleButon, &QPushButton::clicked,
        [model, yeniAlani, durumLabel]() {
            QString yeni = yeniAlani->text().trimmed();
            if (yeni.isEmpty()) return;
```

```

            int satir = model->rowCount();
            model->insertRow(satir);
            model->setData(model->index(satir), yeni);
            yeniAlani->clear();
            durumLabel->setText(
                QString("%1 bölüm kayitli")
                .arg(model->rowCount()));
        });

    // Sil butonu
    QObject::connect(silButon, &QPushButton::clicked,
        [model, liste, durumLabel]() {
            auto secili = liste->currentIndex();
            if (!secili.isValid()) return;
            model->removeRow(secili.row());
            durumLabel->setText(
                QString("%1 bölüm kayitli")
                .arg(model->rowCount()));
        });

    pencere.show();
    return app.exec();
}
```

`setEditTriggers()` ile Düzenleme Kontrolü

`QAbstractItemView::NoEditTriggers` düzenlemeyi tamamen kapatır. `DoubleClicked` ile çift tıklama, `AllEditTriggers` ile her türlü tetikleyici etkinleştirilebilir. Model/View mimarisinde düzenleme modelden bağımsız olarak view üzerinden kontrol edilir.

Görev 1.1 – Çift Tıklama ile Düzenleme (10 dk)

Liste üzerinde düzenlemeyi etkinleştirin:

a) `setEditTriggers()` parametresini `QAbstractItemView::DoubleClicked` olarak değiştirin.

b) Bir bölüm adını çift tıklayarak düzenleyin ve değişikliğin modele yansıdığını doğrulayın.

c) Düzenleme sonrası durum etiketinde `"Duzenlendi: [yeni_ad]"` gösterin. Bunun için modelin `dataChanged` sinyalini kullanın:

```
QObject::connect(model,
    &QStringListModel::dataChanged,
    [model, durumLabel](const QModelIndex& idx) {
        durumLabel->setText(
            QString("Duzenlendi: %1")
            .arg(model->data(idx).toString()));
    });
```

Görev 1.2 – Seçim Bilgisi ve Arama (20 dk)

Liste seçimi ve basit arama ekleyin:

a) Bir öğe seçildiğinde durum etiketinde seçilen bölüm adını ve indeksini gösterin. `QListView`'ın `selectionModel()` üzerinden `currentChanged` sinyalini kullanın:

```
QObject::connect(
    liste->selectionModel(),
    &QItemSelectionModel::currentChanged,
    [durumLabel](const QModelIndex& current) {
        durumLabel->setText(
            QString("Secili [%1]: %2")
            .arg(current.row())
            .arg(current.data().toString()));
    });
```

b) Bir `QLineEdit` ve `"Ara"` butonu ekleyin. Butona tıklandığında modeldeki eşleşen ilk öğeyi seçin:

```
auto eslesme = model->match(
    model->index(0), Qt::DisplayRole,
    aranan, 1, Qt::MatchContains);
if (!eslesme.isEmpty())
    liste->setCurrentIndex(eslesme.first());
```

c) Eşleşme bulunamazsa `QMessageBox::information()` ile `"Sonuc bulunamadi"` mesajı gösterin.

Dikkat: Model Sinyalleri

Model üzerinde `insertRow()` veya `removeRow()` çağrıldığında view otomatik güncellenir – elle güncelleme gerekmez. Bu, Model/View ayrımının en büyük avantajıdır. Ancak `selectionModel()` sinyallerini model oluşturulduktan ve view'a bağlandıktan **sonra** bağlamanız gerekir; aksi halde `selectionModel()` henüz oluşturulmamış olur.

2 Çalışma 2: QAbstractTableModel ile Özel Model (30 dk)

Ön Bilgi

`QAbstractTableModel` sınıfını alt sınıflayarak kendi tablo modelinizi oluşturabilirsiniz. En az üç sanal fonksiyonu geçersiz kılmanız gerekir:

- `rowCount()` – Satır sayısını döndürür
- `columnCount()` – Sütun sayısını döndürür
- `data()` – Belirli bir hücrenin verisini döndürür

Ayrıca `headerData()` ile sütun başlıklarını, `flags()` ile hücre özelliklerini (düzenlenebilir, seçilebilir vb.) özelleştirebilirsiniz.

Bu çalışmada `Q_OBJECT` makrosu kullandığımız için `CMAKE_AUTOMOC ON` gereklidir. Aşağıdaki kodu `main.cpp` dosyasına yazın. Önce öğrenci veri yapısını ve modeli tanımlıyoruz:

```
#include <QApplication>
#include <QMainWindow>
#include <QTableView>
#include <QHeaderView>
#include <QAbstractTableModel>
#include <QVBoxLayout>
#include <QLabel>
#include <QStatusBar>
#include <vector>
#include <QString>

struct Öğrenci {
    int numara;
    QString ad;
    QString bölüm;
    double gpa;
};

class ÖğrenciModel : public QAbstractTableModel {
    Q_OBJECT
public:
    explicit ÖğrenciModel(QObject* parent = nullptr)
        : QAbstractTableModel(parent) {}

    void öğrenciEkle(const Öğrenci& o) {
        beginInsertRows(QModelIndex(),
            m_liste.size(), m_liste.size());
        m_liste.push_back(o);
        endInsertRows();
    }

    int rowCount(const QModelIndex& = {})
        const override {
        return static_cast<int>(m_liste.size());
    }

    int columnCount(const QModelIndex& = {})
        const override {
        return 4;
```

```
    }
```

`data()` fonksiyonu rol bazlı veri döndürür. `Qt::DisplayRole` metin gösterimini, `Qt::TextAlignmentRole` hizalamayı belirler:

```
    QVariant data(const QModelIndex& idx,
        int role = Qt::DisplayRole)
        const override {
        if (!idx.isValid()) return {};
        const auto& o = m_liste[idx.row()];

        if (role == Qt::DisplayRole) {
            switch (idx.column()) {
                case 0: return o.numara;
                case 1: return o.ad;
                case 2: return o.bölüm;
                case 3: return QString::number(
                            o.gpa, 'f', 2);
            }
        }

        if (role == Qt::TextAlignmentRole) {
            if (idx.column() == 0 || idx.column() == 3)
                return int(Qt::AlignCenter);
        }

        return {};
    }
```

`headerData()` ile sütun başlıklarını tanımlıyoruz:

```
    QVariant headerData(int section,
        Qt::Orientation orientation,
        int role = Qt::DisplayRole)
        const override {
        if (role != Qt::DisplayRole) return {};
        if (orientation == Qt::Horizontal) {
            switch (section) {
                case 0: return "No";
                case 1: return "Ad Soyad";
                case 2: return "Bölüm";
                case 3: return "GPA";
            }
        }
        return section + 1;
    }

private:
    std::vector<Öğrenci> m_liste;
};
```

Ana pencereyi oluşturup modeli `QTableView`'a bağlıyoruz:

```
int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QMainWindow anaPencere;
    anaPencere.setWindowTitle("OBYS - Öğrenci Tablosu");
    anaPencere.resize(650, 400);

    auto* model = new ÖğrenciModel(&anaPencere);
```

```
    model->öğrenciEkle({101, "Ali Yilmaz",
        "Bilgisayar", 3.45});
    model->öğrenciEkle({102, "Ayse Kaya",
        "Yazilim", 3.80});
    model->öğrenciEkle({103, "Veli Ozturk",
        "Elektrik", 2.95});
    model->öğrenciEkle({104, "Fatma Demir",
        "Bilgisayar", 1.85});

    auto* tablo = new QTableView;
    tablo->setModel(model);
    tablo->horizontalHeader()
        ->setStretchLastSection(true);
    tablo->setSelectionBehavior(
        QAbstractItemView::SelectRows);

    anaPencere.setCentralWidget(tablo);
    anaPencere.statusBar()->showMessage(
        QString("%1 öğrenci kayitli")
        .arg(model->rowCount()));

    anaPencere.show();
    return app.exec();
}

#include "main.moc"
```

`beginInsertRows()` ve `endInsertRows()`

Modele satır eklerken `beginInsertRows()` ve `endInsertRows()` çağrılmalıdır. Bu fonksiyonlar bağlı view'ları bilgilendirir ve görünümün otomatik güncellenmesini sağlar. Aynı kural silme (`beginRemoveRows`/`endRemoveRows`) için de geçerlidir. `main.moc` dahil etmek, tek dosyalı projelerde `Q_OBJECT` makrosunun çalışması için gereklidir.

Görev 2.1 – Arka Plan Rengi ile GPA Vurgulama (10 dk)

`data()` fonksiyonuna `Qt::BackgroundRole` desteği ekleyin:

a) GPA 2.0'ın altındaysa hücre arka planını açık kırmızı (`QColor(255, 200, 200)`) yapın.

b) GPA 3.5'ın üstündeyse açık yeşil (`QColor(200, 255, 200)`) yapın.

c) Gerekli `#include <QColor>` başlığını eklemeyi unutmayın.

Görev 2.2 – Satır Silme ve Seçim Bilgisi (20 dk)

Modele silme desteği ekleyin ve seçim bilgisini gösterin:

a) `ÖğrenciModel` sınıfına `öğrenciSil(int satir)` metodu ekleyin. `beginRemoveRows()` ve `endRemoveRows()` kullanın:

```
void öğrenciSil(int satir) {
    beginRemoveRows(QModelIndex(), satir, satir);
    m_liste.erase(m_liste.begin() + satir);
    endRemoveRows();
}
```

b) Ana pencereye bir araç çubuğu ekleyin ve `"Sil"` aksiyonu oluşturun. Tıklandığında seçili satırı silin.

c) Satır seçildiğinde durum çubuğunda seçilen öğrencinin adını ve GPA'sını gösterin. `selectionModel()` üzerinden `currentRowChanged` sinyalini kullanın.

Dikkat: Q_OBJECT ve moc

`Q_OBJECT` makrosu içeren sınıflar Meta-Object Compiler (moc) tarafından işlenmelidir. Tek dosyalı projelerde dosya sonuna `#include "main.moc"` ekleyerek moc çıktısını dahil edin. `CMakeLists.txt` dosyasında `set(CMAKE_AUTOMOC ON)` satırının bulunduğundan emin olun.

3 Çalışma 3: Delegeler (30 dk)

Ön Bilgi

Delegeler (`QStyledItemDelegate`) hücrelerin nasıl çizileceğini ve düzenleneceğini kontrol eder. Özel delege oluşturmak için dört sanal fonksiyon geçersiz kılınabilir:

- `createEditor()` – Düzenleme widget'ını oluşturur
- `setEditorData()` – Model verisini editöre aktarır
- `setModelData()` – Editör verisini modele yazar
- `paint()` – Hücreyi özel çizer (ileri seviye)

Delegeler sayesinde bir GPA sütununda `QDoubleSpinBox`, bölüm sütununda `QComboBox` gibi özel düzenleme widget'ları sunabilirsiniz.

Çalışma 2'deki kodu temel alacağız. Önce modele düzenleme desteği ekleyelim. `ÖğrenciModel` sınıfına aşağıdaki fonksiyonları ekleyin:

```
    Qt::ItemFlags flags(const QModelIndex& idx)
        const override {
        if (!idx.isValid())
            return Qt::NoItemFlags;
        return Qt::ItemIsSelectable
             | Qt::ItemIsEnabled
             | Qt::ItemIsEditable;
    }

    bool setData(const QModelIndex& idx,
        const QVariant& deger, int role) override {
        if (!idx.isValid()
            || role != Qt::EditRole) return false;

        auto& o = m_liste[idx.row()];
        switch (idx.column()) {
            case 0: o.numara = deger.toInt(); break;
            case 1: o.ad = deger.toString(); break;
            case 2: o.bölüm = deger.toString(); break;
            case 3: o.gpa = deger.toDouble(); break;
            default: return false;
        }
        emit dataChanged(idx, idx, {role});
        return true;
    }
```

Şimdi GPA sütunu için `QDoubleSpinBox` kullanan bir delege tanımlıyoruz:

```
#include <QStyledItemDelegate>
#include <QDoubleSpinBox>

class GpaDelegate
    : public QStyledItemDelegate {
    Q_OBJECT
public:
    using QStyledItemDelegate::QStyledItemDelegate;

```

```
    QWidget* createEditor(QWidget* parent,
        const QStyleOptionViewItem&,
        const QModelIndex&) const override {
        auto* editor = new QDoubleSpinBox(parent);
        editor->setRange(0.0, 4.0);
        editor->setSingleStep(0.01);
        editor->setDecimals(2);
        return editor;
    }

    void setEditorData(QWidget* editor,
        const QModelIndex& idx) const override {
        double deger = idx.data().toString().toDouble();
        auto* spin =
            static_cast<QDoubleSpinBox*>(editor);
        spin->setValue(deger);
    }

    void setModelData(QWidget* editor,
        QAbstractItemModel* model,
        const QModelIndex& idx) const override {
        auto* spin =
            static_cast<QDoubleSpinBox*>(editor);
        model->setData(idx, spin->value());
    }
};
```

Delegeyi tablonun GPA sütununa atıyoruz:

```
    // main() icinde, tablo oluşturulduktan sonra:
    auto* gpaDelege = new GpaDelegate(tablo);
    tablo->setItemDelegateForColumn(3, gpaDelege);
```

`setItemDelegateForColumn()` Kullanımı

`setItemDelegateForColumn(sutun, delege)` belirli bir sütuna özel delege atar. Tüm tabloya delege atamak için `setItemDelegate()` kullanılır. Satır bazlı atama da `setItemDelegateForRow()` ile mümkündür.

Görev 3.1 – Bölüm Delegesi (10 dk)

Bölüm sütunu için `QComboBox` kullanan bir delege oluşturun:

a) `BolumDelegate` sınıfını `QStyledItemDelegate`'den türetin.

b) `createEditor()` içinde bir `QComboBox` oluşturun ve şu seçenekleri ekleyin: `"Bilgisayar"`, `"Yazilim"`, `"Elektrik"`, `"Makine"`.

c) `setEditorData()` ve `setModelData()` fonksiyonlarını yazın. `QComboBox` için `setCurrentText()` ve `currentText()` kullanın.

d) Delegeyi sütun 2'ye (Bölüm) atayın:

```
auto* bolumDelege = new BolumDelegate(tablo);
tablo->setItemDelegateForColumn(2, bolumDelege);
```

Görev 3.2 – GPA Doğrulama ve Renklendirme (10 dk)

GPA delegesini geliştirin:

a) `setModelData()` içinde GPA değeri 0.0–4.0 aralığında değilse modeli güncellemeyin ve `QMessageBox::warning()` ile uyarı gösterin.

b) `paint()` fonksiyonunu geçersiz kılarak GPA değerine göre hücre arka planını boyayın: kırmızı ($<$ 2.0), sarı (2.0–3.0), yeşil ($>$ 3.0):

```
void paint(QPainter* painter,
    const QStyleOptionViewItem& option,
    const QModelIndex& idx) const override {
    double gpa = idx.data().toString().toDouble();
    QColor renk;
    if (gpa < 2.0) renk = QColor(255, 180, 180);
    else if (gpa < 3.0) renk = QColor(255, 255, 180);
    else renk = QColor(180, 255, 180);

    painter->fillRect(option.rect, renk);
    QStyledItemDelegate::paint(
        painter, option, idx);
}
```

Görev 3.3 – dataChanged Sinyali ile Durum Güncelleme (10 dk)

Model değişikliklerini izleyin:

a) Modelin `dataChanged` sinyalini bir lambda'ya bağlayın.

b) GPA sütunu değiştiğinde durum çubuğunda yeni değeri gösterin: `"GPA güncellendi: [ad] -> [yeni_gpa]"`.

c) Ortalama GPA'yı hesaplayıp durum çubuğunda gösterin: `"Sinif ortalamasi: X.XX"`.

Dikkat: emit dataChanged

`setData()` içinde `emit dataChanged(idx, idx, {role})` çağrısını unutmayın. Bu sinyal olmadan view güncellenmez ve delegenin `paint()` fonksiyonu yeniden çağrılmaz. Üçüncü parametre (`roles`) hangi rollerin değiştiğini belirtir.

4 Çalışma 4: QSortFilterProxyModel (30 dk)

Ön Bilgi

`QSortFilterProxyModel` kaynak modeli değiştirmeden sıralama ve filtreleme sağlayan bir ara modeldir (proxy). Görünüm proxy modeline, proxy modeli de kaynak modele bağlanır:
`KaynakModel` → `ProxyModel` → `View`
Temel özellikleri:

- `setSourceModel()` – Kaynak modeli belirler
- `setFilterFixedString()` – Sabit metin filtresi
- `setFilterRegularExpression()` – Regex filtresi
- `setFilterKeyColumn()` – Filtre uygulanacak sütun
- `setSortRole()` – Sıralama rolü

Tablo başlığına tıklayarak otomatik sıralama için `setSortingEnabled(true)` çağrılır.

Çalışma 2'deki `ÖğrenciModel`'i temel alarak proxy model ekliyoruz. `main()` fonksiyonunda tablo oluşturma kısmını aşağıdaki gibi değiştirin:

```
#include <QSortFilterProxyModel>
#include <QLineEdit>
#include <QComboBox>
#include <QHBoxLayout>

    // ... (ÖğrenciModel ve veri ekleme ayni)

    // Proxy model oluştur
    auto* proxy = new QSortFilterProxyModel(
        &anaPencere);
    proxy->setSourceModel(model);
    proxy->setFilterCaseSensitivity(
        Qt::CaseInsensitive);

    // Tablo proxy modele bağlanır
    auto* tablo = new QTableView;
    tablo->setModel(proxy);
    tablo->setSortingEnabled(true);
    tablo->horizontalHeader()
        ->setStretchLastSection(true);
    tablo->setSelectionBehavior(
        QAbstractItemView::SelectRows);
```

Filtreleme arayüzünü oluşturuyoruz:

```
    // Filtre arayuzu
    auto* merkez = new QWidget;
    auto* anaDuzen = new QVBoxLayout(merkez);

    auto* filtreDuzen = new QHBoxLayout;
    auto* filtreAlani = new QLineEdit;
    filtreAlani->setPlaceholderText(
        "Filtre metni...");
    auto* sutunSecici = new QComboBox;
    sutunSecici->addItems(
```

```
        {"Tumu", "No", "Ad", "Bölüm", "GPA"});
    filtreDuzen->addWidget(
        new QLabel("Filtre:"));
    filtreDuzen->addWidget(filtreAlani);
    filtreDuzen->addWidget(sutunSecici);
    anaDuzen->addLayout(filtreDuzen);

    anaDuzen->addWidget(tablo);
    anaPencere.setCentralWidget(merkez);
```

Filtreleme bağlantılarını kuruyoruz:

```
    // Filtre metni değiştiginde
    QObject::connect(filtreAlani,
        &QLineEdit::textChanged,
        [proxy](const QString& metin) {
            proxy->setFilterFixedString(metin);
        });

    // Sutun secici değiştiginde
    QObject::connect(sutunSecici,
        &QComboBox::currentIndexChanged,
        [proxy](int idx) {
            // 0=Tumu(-1), 1=No(0), 2=Ad(1), ...
            proxy->setFilterKeyColumn(idx - 1);
        });

    anaPencere.statusBar()->showMessage(
        QString("%1 öğrenci kayitli")
        .arg(model->rowCount()));
```

`setFilterKeyColumn(-1)` ile Tüm Sütunlarda Arama

`setFilterKeyColumn(-1)` filtre metnini tüm sütunlarda arar. Belirli bir sütun numarası verilirse (0, 1, 2, ...) yalnızca o sütunda arama yapılır. Bu, OBYS'deki arama çubuğu için ideal bir yaklaşımdır.

Görev 4.1 – GPA Aralık Filtresi (10 dk)

GPA değerine göre özel filtreleme ekleyin:

a) Filtre arayüzüne iki `QDoubleSpinBox` ekleyin: `"Min GPA"` ve `"Max GPA"` (0.0–4.0 aralığı).

b) `QSortFilterProxyModel`'i alt sınıflayarak `filterAcceptsRow()` fonksiyonunu geçersiz kılın:

```
class ÖğrenciProxy
    : public QSortFilterProxyModel {
    Q_OBJECT
public:
    using QSortFilterProxyModel
        ::QSortFilterProxyModel;

    void setGpaAraligi(double min, double max) {
        m_minGpa = min;
        m_maxGpa = max;
        invalidateFilter();
    }

protected:
    bool filterAcceptsRow(int satir,
        const QModelIndex& parent)
        const override {
        // GPA kontrolu (sutun 3)
        auto idx = sourceModel()->index(
            satir, 3, parent);
        double gpa =
            idx.data().toString().toDouble();
        if (gpa < m_minGpa || gpa > m_maxGpa)
            return false;
        // Metin filtresini de uygula
        return QSortFilterProxyModel
            ::filterAcceptsRow(satir, parent);
    }

private:
    double m_minGpa = 0.0;
    double m_maxGpa = 4.0;
};
```

c) SpinBox değerleri değiştiğinde `proxy->setGpaAraligi(min, max)` çağırın.

Görev 4.2 – Filtrelenmiş Sayaç ve Sıralama (10 dk)

Filtreleme sonuçlarını izleyin:

a) Proxy modelin `rowCount()` değerini durum çubuğunda gösterin: `"Gosterilen: X / Y öğrenci"`. Bunun için proxy'nin `layoutChanged` sinyalini kullanın:

```
auto güncelle = [proxy, model, &anaPencere]() {
    anaPencere.statusBar()->showMessage(
        QString("Gosterilen: %1 / %2 öğrenci")
        .arg(proxy->rowCount())
        .arg(model->rowCount()));
};
QObject::connect(proxy,
    &QSortFilterProxyModel::layoutChanged,
    güncelle);
```

b) `"Temizle"` butonu ekleyin: filtre metnini, sütun seçicisini ve GPA aralığını sıfırlasın.

c) Tablo başlığına tıklayarak sıralamanın çalıştığını doğrulayın. GPA sütununa tıklandığında sayısal sıralama yapıldığını kontrol edin (metin olarak değil, sayı olarak sıralama için `data()` fonksiyonunda GPA'yı `double` olarak döndürmeyi düşünün).

Görev 4.3 – Bölüm Filtre Butonları (10 dk)

Hızlı bölüm filtreleme butonları ekleyin:

a) Filtre arayüzünün altına yatay bir `QHBoxLayout` ekleyin.

b) Dört adet `QPushButton` oluşturun: `"Tumu"`, `"Bilgisayar"`, `"Yazilim"`, `"Elektrik"`.

c) Her butona tıklandığında `setFilterKeyColumn(2)` ile bölüm sütununu seçin ve `setFilterFixedString()` ile ilgili bölümü filtreleyin. `"Tumu"` butonu filtreyi temizlesin.

Teslim Bilgileri

Teslim edilecekler:

1. `çalışma1/` – QStringListModel ve QListView (Görev 1.1–1.2)
2. `çalışma2/` – QAbstractTableModel ile özel model (Görev 2.1–2.2)
3. `çalışma3/` – Delegeler (Görev 3.1–3.3)
4. `çalışma4/` – QSortFilterProxyModel (Görev 4.1–4.3)

Her çalışma klasörü bir `CMakeLists.txt` ve `main.cpp` dosyası içermelidir. `CMakeLists.txt` dosyasında `set(CMAKE_AUTOMOC ON)` satırının bulunduğundan emin olun.
Puanlama: Çalışma 1: 25p Çalışma 2: 25p Çalışma 3: 25p Çalışma 4: 25p